

USING ANTIGRAVITY TO BUILD AN ALGORITHMIC TRADING STRATEGY APP

Human-Guided Agentic Development for the
Dow Jones Industrial Average

AGENTIC SOFTWARE ENGINEERING · ALGORITHMIC FINANCE

“The value of an autonomous system lies not in removing human judgement, but in making disciplined experimentation, review and improvement easier to perform.”

ALEJANDRO REYNOSO

JUDGE BUSINESS SCHOOL · UNIVERSITY OF CAMBRIDGE · JULY 2026

Abstract

This report illustrates how Antigravity can be used to generate an appealing application that implements different algorithmic trading strategies for the Dow Jones Industrial Average (DJIA). Starting with a brief natural language prompt, the system was developed through an iterative process involving a human supervisor and autonomous AI agents. The document describes the multi-agent collaboration between the core agent, background task threads, and browser subagent. It traces the continuous improvement loops that led to a dynamic rebalancing feed and timeline playback. Additionally, the appendix serves as a detailed guide for developers to implement the Antigravity algorithmic trading models, complete with mathematical formulations and JavaScript code implementations.

Keywords: autonomous systems; agentic software engineering; algorithmic trading; backtesting; human–AI collaboration; portfolio analytics; financial technology.

Central proposition

Antigravity is most valuable to finance power users when it is treated as a reviewable development partner: the human defines the financial purpose, assumptions and acceptance criteria, while the agent converts those requirements into inspectable data pipelines, models, interfaces, tests and documentation.

Collection note. This monograph forms part of the *Autonomous Systems in Finance* collection, which examines how governed agentic systems can support research, analytical production and decision-oriented software in financial institutions.

Research status. The application is a pedagogical and technical prototype. Its backtests illustrate implementation and review practices; they do not constitute investment advice or evidence of future performance.

Introduction: Antigravity for Finance Power Users

This report is intended as an introduction to Antigravity for power users in finance: analysts, portfolio managers, risk specialists, quantitative researchers, and product leaders who want to turn financial ideas into working software without beginning from a blank codebase. The case study is a Dow Jones Industrial Average (DJIA) algorithmic trading application that lets a user compare different portfolio construction rules, inspect rebalancing behaviour, and watch portfolio composition evolve through time.

The central lesson is that Antigravity is not simply a code generator. It is an agentic development environment in which the human user expresses intent, reviews plans, approves or redirects actions, and evaluates the final product, while the agent translates that intent into files, data pipelines, simulations, tests, and user-interface behaviour. For a finance power user, the practical value is the ability to work at the level of financial logic—strategy design, constraints, data requirements, diagnostics, and interpretation—while still obtaining a functioning application with source code that can be inspected and extended.

The human interaction is therefore part of the architecture. The user begins with an investment objective, observes what Antigravity proposes, corrects the product direction, and asks for refinements such as dynamic portfolio playback. The final product is not a static report or a toy example: it is an appealing browser-based app that implements multiple algorithmic trading strategies, downloads and stores market data, computes backtest metrics, and explains portfolio changes through an interactive timeline. The project progressed through three distinct human-agent phases:

1.1 Pedagogical Orientation for Finance Users

A useful way to understand Antigravity is to compare it with the workflow of a senior finance professional working with an analyst team. The human does not begin by saying exactly which JavaScript function should be written or which CSS selector should control a button. Instead, the human states a goal: build a tool that makes algorithmic allocation across Dow Jones components understandable. Antigravity then behaves like a technical analyst who can plan the work, create the required files, run commands, interpret errors, and revise the application. The crucial difference is that the interaction happens through natural language and through visible artifacts in the workspace.

The human interaction has three layers. First, the user supplies the financial intention. In this project the intention was not simply to draw a chart, but to make an algorithmic trading strategy app that finance users could inspect. Second, the user reviews intermediate outputs. If the first version is too static, the user can ask for animation. If the portfolio changes are hidden, the user can ask for entrants and exits. If the explanation is too technical, the user can ask for a more pedagogical report. Third, the user customizes the final product by changing assumptions, outputs, interface style, and narrative emphasis. Antigravity supports this because the product is not locked inside a black box. It is represented as editable files: data scripts, HTML, CSS, JavaScript, and LaTeX documentation.

For power users in finance, this distinction is important. Many financial tools are either rigid vendor

systems or fragile spreadsheets. Antigravity allows the user to move between conceptual design and implementation. A portfolio manager can say, “I want the strategy to rebalance quarterly instead of monthly.” A risk analyst can say, “Add drawdown and volatility because returns alone are misleading.” A product leader can say, “Make the interface attractive enough for a client demonstration.” Antigravity converts these requests into technical changes while preserving the human’s ability to review the result.

The final product should therefore be read as both an application and a demonstration of a development method. The application implements trading strategies, but the deeper lesson is how a finance user can steer an agentic system. The human provides purpose, context, acceptance criteria, and judgement. Antigravity provides decomposition, coding, testing, iteration, and documentation. A successful workflow requires both sides: the agent can generate code quickly, but only the human can decide whether the code expresses a meaningful financial idea.

1.2 Practical Prompting Principles

For a finance power user, the most important practical skill is learning how to express a request so that Antigravity can transform it into a system. A good prompt states the financial problem, the intended audience, the data universe, the calculations required, and the desired form of interaction. The first prompt in this project was intentionally simple, but later prompts became more product-oriented. This mirrors a realistic workflow: a user may begin with an imperfect idea and then refine it after seeing what the agent produces.

Antigravity works best when the user treats the conversation as a design review. The user should not assume that the first output is final. Instead, the user should ask: What is missing for a practitioner? What would make this credible in front of a portfolio committee? What would make it easier for a student, client, or colleague to understand? These review questions lead naturally to requests for better charts, explicit assumptions, deeper documentation, and visible portfolio transitions.

The system also rewards specificity. If the user says “make it better,” the agent must infer what better means. If the user says “show portfolio entrants and exits at every rebalance,” the agent can identify the data structures and interface elements required. If the user says “make the report pedagogical for power users in finance,” the agent can shift the writing style, define the human role, and explain the internal loop. In this sense, customization is not a separate menu. Customization is performed through precise human feedback.

The final product is therefore a record of increasingly precise human intention. The app begins as a trading strategy idea, becomes a backtester, then becomes an animated explanatory dashboard, and finally becomes a teaching case about Antigravity itself. This sequence shows how power users can start with a practical finance problem and progressively shape an artifact that is useful for demonstration, learning, review, and further development.

1.3 Phase 1: Initial Core Request

The human operator initiated the project with the prompt:

“i want to have an algorithmic strategy to invest in the components of the dow jones”

This prompt was analyzed by the Antigravity core agent, identifying three key components:

- **Data Universe:** The 30 current stocks comprising the Dow Jones Industrial Average.

- ▶ **Strategy Space:** Algorithmic selection and rebalancing methods (e.g., Dogs of the Dow, Momentum, Mean Reversion).
- ▶ **Evaluation Engine:** A backtesting simulator to calculate historical returns, drawdowns, and risks.

The core agent created an implementation plan (`implementation_plan.md`) and a task list (`task.md`), which the user reviewed and approved.

1.4 Phase 2: Transition to Dynamic Playback

After the static codebase was built, the user requested an interactive animation of the backtest:

“now, i want a more interactive visualization showwing the dynamic change in composition of the portfolio...the entrants andd the exiting stocks ... show a dynamic building of the backtesiting courve”

This prompt required a shift from static batch processing to stateful daily simulation. The core agent modified the data structures to cache daily snapshots, added playback controls (play, pause, reset, speed), and built an entrant/exit card system to show which stocks entered and left the portfolio.

1.5 Phase 3: Chronological Timeline Feed

The final request aimed to capture the entire rebalancing history:

“i want to see the changes in the porfolio composition at every rbalancing, in a dynamic display”

The single-event panel was replaced with a scrollable timeline feed. As playback runs, new cards are appended to the feed, building a scrollable record of entrants and exits for all rebalance events since 2015.

How Antigravity Works: Execution Loop and Platform Mechanics

To understand the engineering behind this backtester, it is necessary to examine how Antigravity operates as an agentic software-development system. In this case, the human user did not manually specify every class, function, chart, data schema, or CSS rule. Instead, the user supplied financial intent: build an algorithmic strategy tool for DJIA components, then make the experience more interactive and visually explanatory. Antigravity converted those requests into a sequence of concrete engineering actions.

For finance power users, the important mechanism is the separation between *intent* and *execution*. The human remains responsible for the investment question, acceptable assumptions, interpretation of backtest results, and judgement about whether the application answers a real workflow need. Antigravity handles the operational execution: decomposing the request, writing source files, running commands, checking outputs, and revising the implementation when the user asks for a different final product. This makes the platform useful for rapid prototyping of financial analytics, model dashboards, risk tools, and internal decision-support applications.

Antigravity is an AI-first software engineering ecosystem structured to orchestrate complex development cycles autonomously while keeping the human user in the supervisory loop.

2.1 How the Agent Actually Converts Intent into Work

Antigravity works by turning a human request into a sequence of operational steps. The first step is interpretation. The agent reads the prompt and identifies the objects implied by it. In the phrase “algorithmic strategy to invest in the components of the Dow Jones,” the implied objects are an investable universe, a data source, strategy rules, portfolio accounting, performance metrics, and a user interface. A conventional programming environment would require the user to define these explicitly before work begins. Antigravity can infer an initial structure, present it through a plan or implementation path, and then start building.

The second step is decomposition. The agent separates the project into files and responsibilities. A data acquisition script is responsible for collecting market data. A JSON file stores aligned historical records. An HTML file defines the visible structure. A CSS file defines the visual language. A JavaScript file implements the simulator, charts, controls, and timeline. This decomposition is pedagogically useful because it teaches the finance user how applications are organized. Instead of seeing software as a mysterious object, the user can see a map: data enters through one component, financial rules are calculated in another, and the interface renders the result.

The third step is tool execution. Antigravity is not limited to suggesting code in conversation. It can interact with the local workspace, write files, run commands, compile documents, start servers, and inspect outputs. When a command finishes, the result returns to the agent’s context. If an error appears,

the agent can revise its approach. This reactive loop is central. The agent proposes an action, the environment executes it, and the output becomes evidence for the next decision. In finance terms, this is analogous to a backtesting loop: hypothesis, implementation, measurement, diagnosis, and revision.

The fourth step is review and customization. The user can interrupt the development trajectory at any time. If the app is technically correct but not useful, the user can say so. If the chart exists but does not communicate the portfolio dynamics, the user can request a timeline. If the report is too short or insufficiently pedagogical, the user can require deeper explanation. Antigravity is valuable because it treats these requests as new design constraints rather than as separate projects. It can revise the same artifact repeatedly.

The fifth step is preservation of artifacts. The final product is not merely an answer in a chat window. It remains in the workspace as source code and documentation. This matters for auditability, reproducibility, and learning. A finance power user can inspect the formula for transaction costs, examine how rebalancing dates are detected, or ask another developer to extend the application. Antigravity therefore sits between natural-language ideation and professional software engineering. It accelerates the path from idea to prototype while keeping the implementation visible.

2.2 Why the Execution Loop Matters in Finance

The execution loop matters because finance applications require both speed and discipline. A fast prototype is useful only if the user can inspect and correct it. Antigravity's loop gives the human repeated opportunities to intervene. The user can request a plan before implementation, ask for a narrower change, review modified files, compile the document, or run the application. Each cycle improves the artifact while preserving traceability.

In a financial institution, this style of work can reduce the distance between domain experts and technical implementation. A portfolio professional may understand the strategy but not the browser code. A developer may understand the browser code but not the investment context. Antigravity helps bridge this gap by translating domain language into implementation steps while keeping the resulting code visible. The power user can therefore collaborate with the agent in the same way one might collaborate with a technical team: define requirements, review deliverables, and request revisions.

The loop also supports experimentation. A user can ask Antigravity to create a baseline version, then a version with transaction costs, then a version with a different rebalancing frequency, then a version with a more explanatory interface. Because each revision builds on the previous one, experimentation becomes cheaper. This is particularly valuable in finance, where an idea often needs to be tested across several assumptions before it becomes credible.

The final product benefits from this process because it contains the traces of review. The dynamic timeline exists because the human requested a more intuitive view of portfolio changes. The report exists because the human requested a pedagogical explanation. The bibliography exists because the human wanted references. Antigravity is therefore not only executing code; it is participating in a controlled review process where the user continually raises the standard for the artifact.

2.3 Unified Surfaces and TUI/GUI Channels

The platform exposes three interaction surfaces:

- **Antigravity 2.0 Desktop Application:** An Electron-based desktop surface featuring a side-by-side

split chat canvas, project selector, and an Auxiliary Pane to monitor subagent threads, background tasks, files changed, and active terminals.

- ▶ **Antigravity TUI CLI (agy)**: A terminal-based Text User Interface. It operates via JSON configurations stored in `~/.gemini/antigravity-cli/settings.json` and uses keyboard shortcuts (like ESC to toggle menus or arrow keys for command logs).
- ▶ **Antigravity SDK**: A Python library for leasing virtual agent instances programmatically, invoking remote tools, and orchestrating multi-agent teamwork pipelines.

2.4 The Reactive Agentic Execution Loop

Antigravity operates on a reactive event-driven cycle rather than a polling loop. The execution flow is modeled as follows:

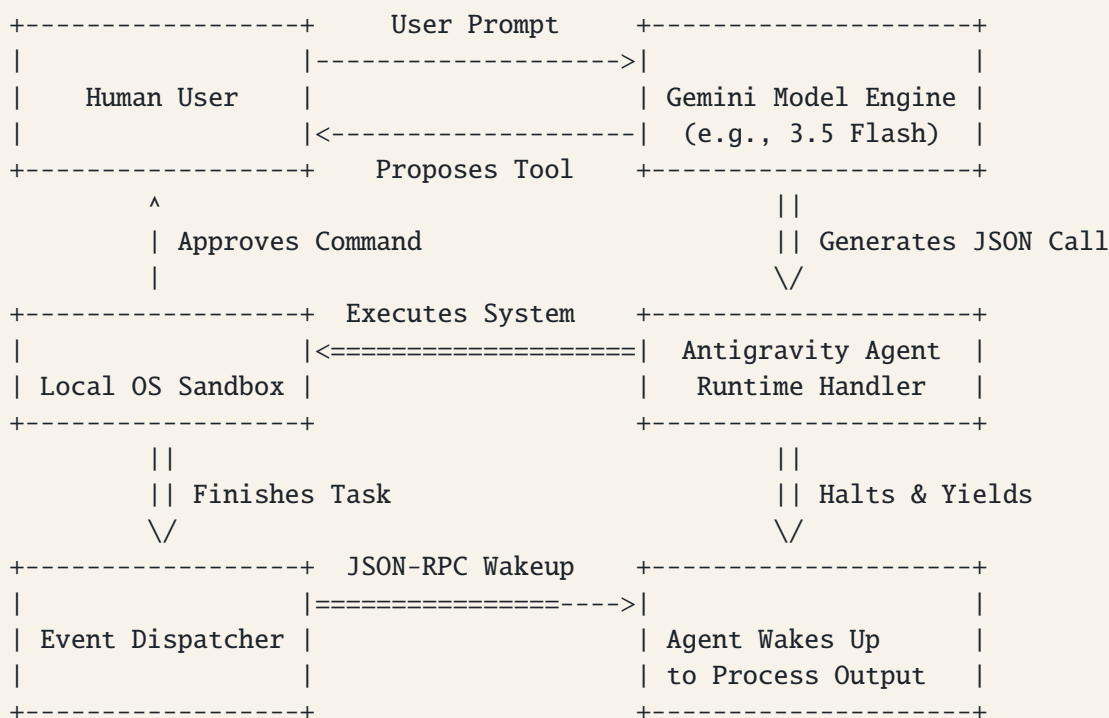


Figure 2.1. Reactive Agentic Loop Cycle

1. **Ingestion & Planning**: The agent receives user input, queries context pools (@ mentions, rule structures), and plans its changes.
2. **Tool Call Proposal**: The agent issues a structured request to run a command, inspect an artifact, or modify a file.
3. **Halt and Yield (Idle State)**: The execution of the agent freezes. No CPU cycles are consumed while the system awaits human approval or background command completions.
4. **Reactive Event Wakeup**: When a background command finishes or the user clicks “Proceed”, the system fires a JSON-RPC notification. The dispatcher wakes the agent, injecting the outputs directly back into its context window.

2.5 Isolation, Permissions, and Sandboxing

Security policies protect the host system during agent runs:

- ▶ **Workspace Restrictions:** By default, file actions are restricted to the active workspace directory. Accessing external directories requires a permission request.
- ▶ **Command Policy Review:** Terminal commands are executed inside a sandboxed environment. Execution modes can be set to `always-proceed`, `request-review`, or `strict` (relying on a trusted command whitelist).
- ▶ **Network Domain Whitelist:** Prevents unauthorized data egress. The agent's web browsing tools can only navigate to domains explicitly permitted by the project settings.

2.6 Skills & Customizations Architecture

Antigravity discovers customizations dynamically from two roots: the Global Customizations Root (`~/ .gemini/config`) and the Workspace Customizations Root (`.agents/`).

- ▶ **Skills (Directory Packages):** Self-contained folders containing a `SKILL.md` instruction file with YAML frontmatter, along with optional helper scripts, templates, and reference materials. These are matched dynamically based on the name and description keys.
- ▶ **Rules (Behavioral Modifiers):** Appended to the `AGENTS.md` markdown file. They define project-specific coding guidelines, constraints, and instructions that are automatically appended to the agent's system instructions.
- ▶ **Model Context Protocol (MCP):** Custom services exposed via local JSON-RPC pipes. They allow developers to expose custom compilers, debuggers, or specialized APIs to the agent's toolbelt.

Agents Involved: Human Supervision and Multi-Agent Architecture

The development and verification of the backtester relied on a coordinated multi-agent structure. The human user acted as the product owner and domain expert: defining the financial use case, judging whether the interface was sufficiently appealing, and requesting changes when the output did not yet communicate portfolio dynamics clearly. Antigravity acted as the implementation coordinator, turning those finance-oriented instructions into code, files, and tests.

This division of labour is central for power users. The user does not need to micromanage every technical step, but should still guide the agent with precise finance language: the investable universe, strategy family, rebalance frequency, benchmark expectations, transaction-cost assumptions, and desired visual diagnostics. The final product emerges from the interaction between these roles: the human defines what matters financially, while the agents build and verify the machinery that makes it usable.

3.1 Roles, Review, and Human Control

The human role in Antigravity is not passive. The user is not merely asking for a finished artifact and hoping that the answer is correct. The user acts as sponsor, reviewer, domain expert, and acceptance tester. In a finance setting, this is essential because correctness is not only syntactic. A program can run without errors and still be financially misleading. For example, a backtest may omit transaction costs, use survivorship-biased data, rebalance unrealistically, or present returns without risk. Antigravity can implement a request, but the human must evaluate whether the request itself is complete.

The agent role is to reduce the cost of experimentation. A human can ask for a change that would normally require several technical steps: modify simulator state, change data structures, update chart rendering, alter layout, and document the new behaviour. Antigravity can perform those steps as one coherent development task. This allows power users to review more alternatives. Instead of accepting the first dashboard, the user can ask for a clearer explanation of rebalancing, a more attractive visual design, additional metrics, or a different strategy rule.

Review happens through artifacts. The user can inspect the LaTeX report, the source files, the compiled PDF, the browser application, and the output of validation commands. This is different from relying on a verbal explanation from the model. The user sees what has actually been changed. If the product does not match the intention, the user can issue another prompt. The agent then uses the existing project state as context and modifies the appropriate files. This continuity is what makes Antigravity suitable for serious workflows.

Customization can occur at several levels. At the financial level, the user can change the universe, strategy, rebalance frequency, transaction cost, benchmark, and performance metrics. At the interface level, the user can request new controls, animations, labels, colour schemes, and explanatory cards. At

the governance level, the user can ask for assumptions to be documented, limitations to be disclosed, and calculations to be made transparent. At the deployment level, the user can request local execution, static hosting, or integration into a larger internal workflow. Antigravity supports customization because it works directly with the project files rather than only with abstract descriptions.

- ▶ **Antigravity Core Agent:** The primary coordinator. It processes user prompts, makes architectural design choices, plans execution paths, manages the filesystem, writes source code, and coordinates subagents.
- ▶ **Browser Subagent:** A specialized agent designed to verify web layouts. It is equipped with tools to launch a Chromium instance, navigate pages, read the DOM tree, type input, click elements, and capture screenshots. During Phase 1 verification, this subagent reported a local Chrome execution failure because the Windows environment restricts the sandbox mode. Antigravity core handled this failure by pivoting to automated unit testing and manual user inspection.
- ▶ **Task Executors (OS/Process Layer):** Background execution nodes managing parallel execution commands, such as installing packages via `pip` or serving local web assets via Python's HTTP server.

3.2 From Natural Language to Files

A key lesson for power users is that Antigravity turns natural language into file-level work. When the user requests an application, the agent must decide which artifacts are required. The request becomes a data script, a dataset, an interface, a simulator, style rules, and documentation. This is why the user can review the final product at multiple levels. If the visual design is unsatisfactory, the CSS can be changed. If the strategy calculation is unsatisfactory, the JavaScript can be changed. If the data source is unsatisfactory, the data pipeline can be changed.

This file-level transparency is one of the reasons Antigravity is useful for finance. Many financial prototypes are difficult to review because assumptions are hidden in spreadsheets, notebooks, or vendor settings. In this project, the assumptions are represented in source files and described in the report. The user can ask Antigravity to explain any part of the implementation, change it, or add a validation step. The agent's work is not merely generative; it is inspectable.

The workflow also shows how Antigravity handles dependencies between tasks. The app cannot compute a strategy without data. It cannot animate portfolio changes unless the simulator stores historical states. It cannot show entrants and exits unless it compares current and previous holdings. It cannot explain itself unless the report describes both the human interaction and the implementation. Antigravity links these requirements together and modifies the necessary parts of the project.

For power users, the practical advice is to describe both the desired output and the reason for it. Saying "show entrants and exits" is good; saying "show entrants and exits so that the user understands portfolio turnover at every rebalance" is better. The reason helps the agent choose an appropriate design. It may create timeline cards rather than a raw table, because the purpose is explanation rather than only storage.

3.3 Pedagogical Summary of the Agent Team

A simple mental model is to think of Antigravity as a coordinated project room. The human sets the agenda. The core agent manages the work. Specialist tools and subprocesses perform tasks such as running commands, validating files, or checking output. The product is improved when the human treats this project room seriously: giving clear requirements, reviewing artifacts, and asking for targeted

revisions.

In finance, this coordination is valuable because projects often require several kinds of expertise at once. A backtester needs investment logic, data engineering, user-interface design, numerical validation, and documentation. Antigravity can move across these layers faster than a single human switching manually between them. However, the human remains responsible for deciding what is financially sensible. The agent can implement a dividend-yield strategy, but the human must decide whether that strategy belongs in the demonstration and how it should be explained.

The user can also customize the agent team's behaviour by changing the request. A request for a teaching report activates a different style of work from a request for a production API. A request for a more appealing app activates design work. A request for more careful assumptions activates documentation and validation work. This means the human is continuously shaping not only the product, but also the way the agent allocates attention.

Because Antigravity operates on editable artifacts, review can be concrete. The user can read the generated report, inspect code, run the app, and ask for corrections. This is different from a one-shot chatbot answer. The agent team can be redirected after evidence appears. If a chart is confusing, the human can ask for another representation. If a section is too short, the human can ask for a deeper explanation. If a calculation needs a caveat, the human can ask for documentation. The final product becomes stronger because the human-agent team repeatedly converts review into implementation.

Agent Workflow: From Human Prompt to Working Finance Application

This section explains how Antigravity moved from a high-level human prompt to a working application. The process is useful for finance power users because it shows what kinds of instructions produce concrete outputs. A vague instruction such as “build a trading app” can start the process, but the most valuable refinements come when the human specifies the desired financial workflow: compare strategies, show entrants and exits, animate the backtest curve, and expose rebalancing events.

Antigravity’s workflow combined planning, environment discovery, data acquisition, interface construction, simulation logic, and validation. At each stage, the final product became more tangible: first a data-backed backtester, then an interactive dashboard, and finally a dynamic explanatory tool for understanding how algorithmic rules change the portfolio over time. The agents took the following steps to build and verify the system:

4.1 What Power Users Should Learn from the Workflow

The workflow demonstrates a practical method for using Antigravity. The first lesson is to begin with the business or investment purpose, not with code. A strong prompt identifies the financial decision the application should support. In this case, the decision is how to evaluate and compare systematic allocation strategies across Dow Jones constituents. The second lesson is to specify the evidence the user wants to see. A static return number is not enough; the user wanted to see the dynamic construction of the backtest curve and the changing composition of the portfolio. That request forced the application to become more explanatory.

The third lesson is to ask for iteration in plain language. The user did not need to know that the simulator had to cache daily snapshots or that the timeline required DOM manipulation. The user described the desired experience: show entrants and exiting stocks, show composition at every rebalance, and make the display dynamic. Antigravity inferred the implementation implications. This is one of the main advantages for finance power users: they can describe the analytical workflow, and the agent can translate that workflow into technical requirements.

The fourth lesson is to validate the output. Antigravity can run scripts and compile documents, but the human should still ask whether the output makes sense. Are there enough observations? Are prices missing? Are dividends handled correctly? Does the strategy rebalance when expected? Are transaction costs applied to turnover rather than to total portfolio value? These questions are finance questions. They are also software quality questions. The best use of Antigravity combines both.

The fifth lesson is to document the final product. Documentation is not an afterthought. A finance app is easier to trust when the report explains how it was built, what assumptions it uses, how the human guided the agent, and what the final files do. Antigravity can help produce that documentation in LaTeX,

but the human must make sure the narrative is appropriate for the audience. In this report, the intended audience is not only programmers. It is power users in finance who want an intuitive explanation of how agentic development can support analytical application building.

1. **Path & Runtime Analysis:** Proposed an environment check to locate Python and confirmed a local Python 3.14.3 installation suitable for the data and validation scripts.
2. **Asynchronous Dependency Ingestion:** Proposed a background command to install dependencies:

```
python -m pip install pandas yfinance
```

The task system executed this, installing `pandas 3.0.3`, `yfinance 1.5.1`, and `numpy 2.5.1`.

3. **Data Universe Harvesting:** Wrote and executed `download_data.py`. The task downloaded daily prices and dividend events for all 30 DJIA components (from 2015-01-01 to 2026-07-13) and saved them to `djia_history.json`.
4. **Dashboard Implementation:** Built the HTML interface, CSS design system, and JavaScript backtesting engine.
5. **Validation:** Created `verify_backtest.py` to check data integrity, confirming 2,897 days of stock history with no NaNs or negative prices.
6. **Spawning Local Delivery:** Started a local web server:

```
python -m http.server 8000
```

which serves the backtester on port 8000.

4.2 Review as a Design Discipline

Review is not a final administrative step; it is a design discipline. In Antigravity, the human improves the product by reacting to what the agent has built. This is especially useful in finance because many requirements become obvious only after a prototype exists. A user may initially think they need a return chart, but after seeing it may realize that the chart does not explain why the strategy behaved as it did. That realization leads to a better product requirement: show the portfolio composition and rebalancing events.

The review process should be structured. A power user can review the application from the standpoint of financial logic, user experience, and governance. Financial logic asks whether the calculations reflect the intended model. User experience asks whether the interface makes the model understandable. Governance asks whether assumptions, limitations, and risks are visible. Antigravity can address all three, but only if the human asks for them or identifies the gap during review.

Improvement is cumulative. A request to animate the backtest requires the simulator to expose intermediate states. A request to show every rebalance requires those states to be compared over time. A request to write a pedagogical report requires the implementation to be explained in accessible language. Each improvement adds a layer of usefulness. The final product is not merely more complex; it is more communicative.

This is why Antigravity is powerful for finance education and professional prototyping. It allows the user to demonstrate a concept, critique it, and refine it quickly. The user learns not only from the final app but from the path of revisions. The process teaches what assumptions matter, what visualizations help, and what documentation is needed for a serious audience.

4.3 Customization During the Workflow

Customization during an Antigravity workflow is incremental. The user can start with a simple application, then ask for one improvement at a time. This is often better than trying to specify everything upfront. In finance, requirements become clearer after the first output is visible. A user may discover that the most important feature is not another statistic, but a better explanation of portfolio turnover. Another user may discover that the interface needs export buttons or assumptions panels.

Antigravity supports this incremental process because it keeps context from the existing workspace. It can see the current files, infer how the pieces connect, and apply a focused change. If the user asks to improve the timeline, the agent can find the rendering logic. If the user asks to improve the report, the agent can edit the LaTeX. If the user asks to compile the result, the agent can run LaTeX and inspect errors. This makes customization feel like a conversation with a technical team rather than a sequence of isolated code-generation requests.

The review process should ask whether the product now better serves the financial task. More code is not automatically better. Better means clearer assumptions, more relevant metrics, more understandable output, and a workflow that helps the target user make sense of the strategy. Antigravity allows the user to make those judgements repeatedly and then request improvements directly.

A practical workflow for a power user is therefore: state the finance objective, ask Antigravity for a plan, review the planned artifacts, allow implementation, run or compile the output, critique the result, and request specific refinements. The user can repeat this sequence until the product is clear enough for the intended audience. This is how a rough idea becomes a usable finance application rather than remaining a generic demonstration.

Iterative Development: Human Feedback as Product Design

Antigravity development is iterative. The first version of an application is rarely the final version, especially in finance where users often discover what they need only after seeing a prototype. In this case, the human interaction changed the product from a static backtesting calculator into a more appealing decision-support application. The user asked not only for performance numbers, but also for a visual explanation of how the portfolio composition changed at each rebalance.

This is an important pattern for power users: use Antigravity to create an initial working artifact, then critique it as if reviewing a junior analyst's prototype. Ask for better diagnostics, clearer charts, more transparent assumptions, and interaction patterns that support financial reasoning. The system went through three major software versions:

5.1 Customization, Improvement, and Review Cycles

The iterative cycle is the heart of Antigravity. A first prompt establishes direction, but subsequent prompts create quality. In this project, the first version solved the basic problem of implementing a backtester. The later versions solved the more important communication problem: how to make the portfolio's behaviour understandable. This is common in finance. The initial analytical model may be correct, but the product is not yet useful until it helps a decision-maker see what is happening.

Antigravity allows customization because each user request can target a different layer of the system. A user can ask for a change in computation, such as adding a momentum strategy or altering the rebalancing calendar. The agent then modifies the simulator. A user can ask for a change in presentation, such as adding cards that show which tickers joined or left the portfolio. The agent then modifies the interface and rendering logic. A user can ask for a change in explanation, such as making the report more pedagogical. The agent then modifies the LaTeX documentation. These are not disconnected activities; they are coordinated edits to the same product.

Improvement also comes from comparison. After seeing one version, the human can compare it with the desired experience. If the app looks too technical, the user can ask for a more appealing interface. If the output hides assumptions, the user can ask for explicit documentation. If the calculation is too simplistic, the user can ask for transaction costs or risk metrics. Antigravity's advantage is speed: the cost of asking for a better version is low, so the user is encouraged to review critically.

Review should be deliberate. A finance power user should examine three questions after each iteration. First, does the application implement the intended financial logic? Second, does the interface help the target audience understand the logic? Third, does the documentation disclose enough assumptions and limitations? Antigravity can assist with all three, but the user remains accountable for accepting the final product. This is especially important in finance, where tools influence decisions involving capital, risk, and client communication.

5.2 Version 1.0: Static Batch Engine

The initial implementation ran backtests on page load. It calculated cumulative statistics and rendered final static plots. It did not track daily asset allocation changes or individual trade histories.

5.3 Version 2.0: Caching Playback Engine

Refactored the simulator to record daily states. This allowed the system to:

- ▶ Track a daily `dailySnapshots` array with asset shares, cash, and total valuation.
- ▶ Update the `Chart.js` dataset slices on every tick of a timer loop.
- ▶ Detect when the playback index crossed a rebalancing index, calculating entrants and exits relative to the previous rebalance date.

5.4 Version 3.0: Chronological Timeline Feed

Replaced the single-event display with a scrollable feed container (`#timelineFeed`).

- ▶ **Dynamic Appending:** During playback, when a rebalance date is crossed, a new event card is created and prepended to the top of the feed container.
- ▶ **Static Rendering:** When running a new backtest, the full timeline is rendered immediately. Prepending chronological cards ensures the latest rebalance event is always visible at the top.
- ▶ **Animation Optimization:** Calling `chart.update('none')` disabled default transitions during playback. This prevented `Chart.js` from lagging on daily data updates.

5.5 Why Iteration Produces a Better Finance Tool

Iteration produces a better finance tool because financial usefulness depends on context. A technically correct model may still fail if users cannot understand its assumptions. A visually attractive dashboard may still fail if the calculations are weak. A long report may still fail if it does not explain the human-agent interaction. Antigravity helps reconcile these needs by allowing repeated changes across computation, design, and explanation.

The user can review each version as a product owner. Version 1 answers the question, “Can the strategy be simulated?” Version 2 answers, “Can the path of the simulation be shown?” Version 3 answers, “Can the portfolio changes be explained as they occur?” The present report answers, “Can this case teach power users how Antigravity works?” Each version has a different purpose, and Antigravity adapts the artifact accordingly.

This is the practical meaning of improvement. The product becomes more aligned with the user’s audience. A student needs intuition. A portfolio committee needs assumptions and risk metrics. A developer needs file structure and code. A compliance reviewer needs transparency and documentation. Antigravity allows the human to request improvements for each of these audiences without rebuilding the project from zero.

Final Product: Appealing Algorithmic Trading Strategy App

The final product is a local browser application for exploring algorithmic trading strategies on the DJIA universe. It is designed to be appealing not merely in visual terms, but in analytical terms: the user can connect a strategy rule to its portfolio consequences, watch value evolve through time, and inspect which stocks enter or leave during rebalancing. This matters for finance users because a backtest is only useful if its assumptions and mechanics are legible.

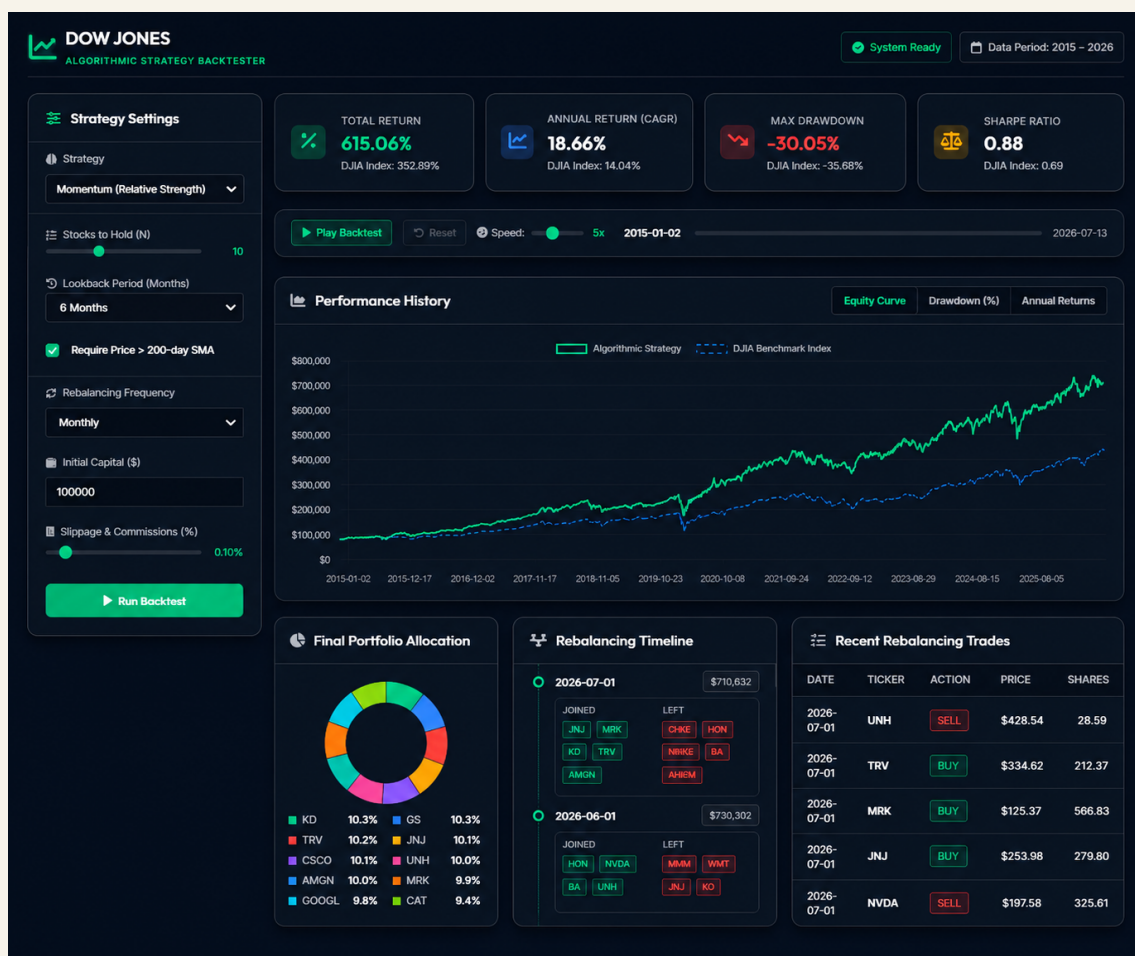


Figure 6.1. Screenshot of the Antigravity-generated algorithmic trading strategy application, showing the interactive interface used to explore DJIA portfolio strategies and dynamic backtest results.

The human interaction produced a product with three layers. The first layer is data: historical prices and dividends are gathered and aligned. The second layer is financial logic: strategy rules select securities, rebalance portfolios, apply transaction costs, and calculate performance metrics. The third layer is experience: controls, charts, and timeline cards make the model understandable to a non-developer stakeholder. The final application resides in a workspace directory containing five core files:

6.1 What Makes the Product Appealing and Useful

An appealing finance application is not simply colourful. It is appealing because it reduces cognitive friction. The user can understand what the strategy does, what inputs it uses, what outputs it produces, and how to interpret changes over time. The Antigravity-built app aims to do this by combining controls, performance charts, strategy rules, and rebalancing explanations in one coherent experience.

The final product is also useful because it supports customization. A user can ask Antigravity to add a new strategy, change the investment universe, alter the styling, create explanatory tooltips, export results, or include additional risk measures. Because the application is built from ordinary files, each customization can be implemented directly. The user is not dependent on a closed vendor interface or a fixed template.

Review remains part of the product's life cycle. After the app runs, the user should compare its outputs with expectations. If a strategy turns over too frequently, the user may request a turnover metric. If performance looks strong but volatile, the user may request rolling drawdowns. If the interface is intended for teaching, the user may request plain-language explanations beside each control. Antigravity can continue improving the product in response to these review findings.

The final product therefore functions as a reusable blueprint. It demonstrates how to combine data acquisition, algorithmic selection, portfolio accounting, visualization, and documentation. A finance user can adapt the same structure to other asset classes, other investment rules, or other decision-support settings. The deeper value is not only the DJIA backtester itself, but the method for building and refining finance applications through human-agent collaboration.

6.2 Interpreting the Final Product as a Finance Power-User Template

The final application can be used as a template for other finance projects. The same Antigravity workflow could build a credit-risk dashboard, a private-equity screening tool, a factor-investing explorer, an insurance pricing prototype, or a regulatory monitoring interface. The pattern is transferable: define the financial problem, specify the data, describe the decision logic, request the interface, review the output, and iterate.

The app is appealing because it joins computation with explanation. A user can see not only the final return but also the sequence of portfolio changes that produced it. This matters because algorithmic strategies often fail to persuade stakeholders when they appear as opaque rules. By showing entrants, exits, rebalancing dates, and portfolio value over time, the application turns a rule-based strategy into something inspectable. That inspectability is also a governance feature. The user can ask why a stock entered, whether the rule was applied consistently, and how much the portfolio changed at each event.

Antigravity enables future customization. A finance user could ask to add a benchmark such as the DIA ETF, export results to CSV, include sector constraints, compare equal-weight versus capitalization-weight allocations, add a drawdown chart, or create a management summary page. The agent would then modify the relevant files. This is why the final product should not be viewed as closed. It is a living prototype that can continue to evolve under human direction.

For review, the final product offers multiple checkpoints. The data file can be inspected for completeness. The formulas can be compared with standard definitions. The JavaScript functions can be read directly. The visual output can be evaluated by users. The LaTeX report can explain the architecture. This multi-layer review process is the practical meaning of using Antigravity responsibly. The agent accelerates

production, but the human reviews substance, clarity, and suitability.

1. `download_data.py`: Downloads prices and dividends and writes the aligned historical dataset.
2. `djia_history.json`: Aligned 2,897-day database of raw close prices, adjusted close prices, and dividends.
3. `index.html`: Semantic HTML5 structure with custom widgets for controls and the timeline.
4. `style.css`: Custom design system utilizing HSL variables for emerald, blue, and slate tones, glassmorphism card blur, and CSS animations.
5. `app.js`: Simulator engine, playback timeline loop, and Chart.js renderer.

The application runs locally using Python's built-in HTTP server:

```
python -m http.server 8000
```

Navigate to `http://localhost:8000/` to run the application.

6.3 How Users Can Extend the Final Product

A power user can extend the final product in many directions. At the strategy level, the user might add low-volatility selection, quality screens, moving-average crossovers, sector caps, or benchmark-relative constraints. At the risk level, the user might add rolling volatility, beta, tracking error, turnover, or value-at-risk. At the presentation level, the user might add tooltips, export buttons, scenario summaries, or a management-facing executive page. Each extension can be requested in natural language and then reviewed in the source files.

Antigravity makes these extensions practical because it understands the relationship between files. Adding a new metric may require a calculation in JavaScript, a display location in HTML, styling in CSS, and explanation in the LaTeX report. A human power user can state the desired capability; Antigravity can coordinate the technical edits. The user then reviews whether the implementation is faithful to the financial idea.

The final product should also be reviewed for limitations. It is a prototype and teaching artifact, not an investment recommendation. It depends on historical data, simplified assumptions, and selected strategy rules. Antigravity can help document these limitations clearly. This is part of responsible customization: improving an application means not only adding features, but also making the scope and assumptions understandable.

As a template, the product demonstrates a repeatable pattern. State the finance problem, build a minimal version, review the result, ask for deeper interaction, validate the calculations, and document the system. This pattern is useful for any power user who wants to use Antigravity as a practical bridge between financial expertise and working software.

The product is also pedagogical because it exposes the connection between a financial instruction and a technical artifact. A user can see that “rebalance quarterly” becomes date logic, that “show entrants and exits” becomes comparison of asset sets, and that “make the app appealing” becomes layout, colour, animation, and explanatory copy. This is how Antigravity teaches by building. The user learns software architecture through the finance problem, not through an abstract programming exercise.

Finally, the product remains open to review. The user can ask Antigravity to explain any function, summarize any file, or rewrite any section of the report. This matters because finance work often requires communication to different audiences. A technical appendix may satisfy developers, while a short narrative may satisfy executives. Antigravity can help maintain both versions as the application evolves.

6.4 Review Checklist for the Finished App

A careful finance user should review the finished app with a checklist. First, check the economic question: does the application actually help compare algorithmic trading strategies, or does it only display attractive graphics? Second, check the data question: are the dates, prices, dividends, and ticker universe appropriate for the exercise? Third, check the modelling question: are rebalancing, target weights, and transaction costs implemented in a way that matches the stated assumptions? Fourth, check the communication question: can a reader understand why the portfolio changes through time? Fifth, check the governance question: are limitations and assumptions visible enough for a professional audience?

Antigravity supports this checklist because every item can become a follow-up request. If the data question is weak, the user can ask for a validation script or a data-quality paragraph. If the modelling question is weak, the user can ask for a corrected formula or additional metric. If the communication

question is weak, the user can ask for labels, tooltips, timeline cards, or rewritten documentation. The final product is therefore not a fixed endpoint. It is a reviewed and reviewable artifact that can keep improving under human control.

This review capacity is what makes Antigravity especially useful for power users in finance. The user can move from idea to prototype quickly, but can also slow down and inspect the important details. Speed and review are not opposites in this workflow. The agent accelerates implementation, while the human raises the standard for clarity, correctness, and usefulness.

AUTONOMOUS SYSTEMS IN FINANCE

APPENDICES

Models, implementation details and prompting record

Algorithmic Trading Model Implementation Guide

This appendix provides a step-by-step mathematical and programmatic guide to the trading models implemented in the Antigravity-built application. For finance power users, this section is also a guide to communicating with Antigravity: each formula corresponds to a requirement that can be stated in natural language and then translated into code. For example, asking for transaction costs requires a model of turnover; asking for a momentum strategy requires a lookback window and ranking rule; asking for a timeline requires the simulator to store intermediate portfolio states rather than only final results.

The final product combines financial assumptions with executable JavaScript. Antigravity's role is to maintain consistency between the conceptual model, the data structures, and the interface. The human user's role is to challenge whether the assumptions are economically meaningful and whether the outputs are appropriate for the intended use.

A.1 How to Read this Appendix as an Antigravity Lesson

This appendix is not only a mathematical appendix. It is a demonstration of how Antigravity connects financial language to implementation. Each formula represents a translation step. When the human says “portfolio value,” Antigravity needs holdings, prices, and cash. When the human says “rebalance,” Antigravity needs a calendar rule, target weights, current holdings, and order calculations. When the human says “show performance,” Antigravity needs CAGR, volatility, Sharpe ratio, and drawdown. When the human says “make it understandable,” Antigravity needs charts, labels, cards, and explanatory text.

For power users, the appendix teaches how to prompt more effectively. Instead of asking only for “a backtester,” the user can ask for specific components: “Use adjusted close prices for valuation,” “rank stocks by six-month momentum,” “apply a transaction cost to turnover,” “store daily portfolio snapshots,” and “display entrants and exits at every rebalance.” The more clearly the user states the financial rule, the more directly Antigravity can implement it. However, the user does not need to write the code. The agent supplies the technical expression.

The appendix also supports review. Because the formulas are written down, the human can check whether the implementation matches the intended financial logic. Because the code snippets are included, the user can see how the formulas become executable instructions. This closes the loop between explanation and product. In a finance environment, that loop is essential. A tool that cannot explain itself is difficult to trust; a tool whose calculations are visible can be challenged, improved, and governed.

A.2 Mathematical Formulations

The mathematical layer shows how a human finance requirement becomes an executable model. In Antigravity, the user can describe these requirements in domain language—portfolio value, rebalancing,

transaction costs, yield ranking, momentum ranking, drawdown, and Sharpe ratio—and the agent can translate them into formulas and code. The human interaction remains essential because the user must decide which assumptions are appropriate for the investment question and which metrics are persuasive for the intended audience.

The final product uses these formulas to make the application more than a visual prototype. Every chart, card, and metric is grounded in a reproducible calculation that can be audited, modified, or extended by a finance power user.

A.2.1 Portfolio Valuation Model

This section explains the accounting core of the final product. Antigravity must represent a portfolio in a way that finance users recognize: holdings, prices, cash, and total value. The human interaction defines what should be included in the valuation, while the agent implements the repeatable calculation. Let C_t be the cash balance on trading day t . Let $S_{i,t}$ be the number of shares held of stock i on day t . Let $P_{i,t}^{adj}$ be the adjusted close price of stock i on day t (accounting for splits and dividend distributions). The total portfolio value V_t on day t is:

$$V_t = C_t + \sum_{i \in \mathcal{U}} S_{i,t} \cdot P_{i,t}^{adj} \quad (\text{A.1})$$

where $\mathcal{U}$ is the universe of 30 DJIA tickers.

A.2.2 Trading Costs and Rebalancing

This section shows how Antigravity converts a realistic finance constraint into code. A power user would not want a backtest that ignores turnover, because an attractive strategy can disappear once trading costs are included. The human specifies the rebalance frequency and cost assumption; Antigravity implements the accounting. Rebalancing occurs at specific daily indices $t \in \mathcal{T}_{rebal}$. Let $w_{i,t}$ be the target portfolio weight of asset i on rebalance day t , where $\sum_{i \in \mathcal{U}} w_{i,t} = 1.0$. The target dollar allocation for stock i is $V_{i,t}^{target} = V_t \cdot w_{i,t}$. The current value of shares held is $V_{i,t}^{curr} = S_{i,t-1} \cdot P_{i,t}^{adj}$. The trade amount $|\Delta V_{i,t}|$ for stock i is:

$$|\Delta V_{i,t}| = |V_{i,t}^{target} - V_{i,t}^{curr}| \quad (\text{A.2})$$

Given a transaction cost rate γ (e.g., 0.001 for 0.10% slippage/commission), the total transaction costs TC_t are computed as:

$$TC_t = \gamma \cdot \sum_{i \in \mathcal{U}} |\Delta V_{i,t}| \quad (\text{A.3})$$

The remaining investable capital V_t^{net} is:

$$V_t^{net} = V_t - TC_t \quad (\text{A.4})$$

New share quantities $S_{i,t}$ are updated as:

$$S_{i,t} = \frac{V_t^{net} \cdot w_{i,t}}{P_{i,t}^{adj}} \quad (\text{A.5})$$

The cash balance becomes:

$$C_t = V_t^{net} - \sum_{i \in \mathcal{U}} S_{i,t} \cdot P_{i,t}^{adj} \quad (\text{A.6})$$

A.2.3 Dogs of the Dow (Dividend Yield Selection)

This strategy demonstrates how Antigravity can implement a named finance rule rather than a generic machine output. The human asks for a recognizable strategy family; the agent expresses it as ranking logic, data requirements, and portfolio weights. For rebalancing date t , the trailing 12-month dividend yield $Y_{i,t}$ for stock i is calculated using the raw close price $P_{i,t}^{close}$ and the sum of dividends paid in the last 365 days:

$$Y_{i,t} = \frac{\sum_{\tau \in [t-365, t]} Div_{i,\tau}}{P_{i,t}^{close}} \quad (A.7)$$

The assets are ranked in descending order of yield:

$$Y_{(1),t} \geq Y_{(2),t} \geq \dots \geq Y_{(|\mathcal{U}|),t} \quad (A.8)$$

The top N stocks are selected ($\mathcal{S}_t = \{(1), (2), \dots, (N)\}$). Weights are assigned equally:

$$w_{i,t} = \begin{cases} \frac{1}{N} & \text{if } i \in \mathcal{S}_t \\ 0 & \text{otherwise} \end{cases} \quad (A.9)$$

A.2.4 Momentum (Relative Strength Selection)

Momentum illustrates the same human-agent pattern with a different investment belief: recent relative winners may continue to outperform. The human chooses the lookback window and number of holdings; Antigravity implements the return calculation and sorting rule. Let L represent the lookback period in trading days (e.g., 6 months ≈ 126 days). The momentum return $R_{i,t}^M$ is:

$$R_{i,t}^M = \frac{P_{i,t}^{adj} - P_{i,t-L}^{adj}}{P_{i,t-L}^{adj}} \quad (A.10)$$

An optional 200-day Simple Moving Average (SMA) filter requires:

$$P_{i,t}^{adj} > \text{SMA}_{200}(P_i^{adj})_t = \frac{1}{200} \sum_{\tau=t-199}^t P_{i,\tau}^{adj} \quad (A.11)$$

Stocks failing the SMA check are filtered out of the selection pool. The remaining assets are sorted in descending order of $R_{i,t}^M$, and the top N are assigned equal weights $1/N$.

A.2.5 Mean Reversion Selection

Mean reversion illustrates how a contrasting hypothesis can be added to the same application. The final product becomes more useful because the user can compare competing financial intuitions inside one interface. Calculated identically to Momentum, except the selection pool is sorted in ascending order of $R_{i,t}^M$ (worst performers selected).

A.2.6 CAGR (Compound Annual Growth Rate)

CAGR converts the simulated wealth path into a familiar finance metric. Antigravity computes it automatically, but the human user interprets whether the result is meaningful after considering risk,

turnover, and benchmark context.

$$\text{CAGR} = \left(\frac{V_{\text{end}}}{V_{\text{start}}} \right)^{\frac{252}{D_{\text{total}}}} - 1 \quad (\text{A.12})$$

where D_{total} is the total number of simulated trading days.

A.2.7 Annualized Volatility

Volatility adds the risk dimension that finance users need before treating any return result as attractive. The human asks for risk diagnostics; Antigravity derives them from the daily valuation series. Daily returns are calculated as $r_t = \frac{V_t - V_{t-1}}{V_{t-1}}$. Given the mean daily return $\bar{r}$, daily variance σ_{daily}^2 is:

$$\sigma_{\text{daily}}^2 = \frac{1}{D_{\text{total}} - 1} \sum_{t=1}^{D_{\text{total}}-1} (r_t - \bar{r})^2 \quad (\text{A.13})$$

Annualized volatility σ_{annual} is:

$$\sigma_{\text{annual}} = \sqrt{\sigma_{\text{daily}}^2} \cdot \sqrt{252} \quad (\text{A.14})$$

A.2.8 Sharpe Ratio

The Sharpe ratio shows how the final product can combine user-defined assumptions with standard performance evaluation. A power user can change the risk-free rate or compare strategies under a consistent assumption. Assuming an annualized risk-free rate R_f (defaulting to 2.0%):

$$\text{Sharpe Ratio} = \frac{\text{CAGR} - R_f}{\sigma_{\text{annual}}} \quad (\text{A.15})$$

A.2.9 Maximum Drawdown

Maximum drawdown makes the app more useful for practitioners because it exposes path dependency and downside experience, not only endpoint returns. This is the type of metric a human reviewer should request when transforming a prototype into a decision-support tool. Let $H_t = \max_{\tau \leq t} V_\tau$ represent the peak portfolio valuation up to day t . The drawdown DD_t is:

$$DD_t = \frac{V_t - H_t}{H_t} \quad (\text{A.16})$$

The maximum drawdown is:

$$\text{Max Drawdown} = \min_t DD_t \quad (\text{A.17})$$

A.3 JavaScript Code Implementations

The following sections provide code snippets from the simulation engine to show how these mathematical formulas are implemented programmatically. This is where Antigravity's value becomes concrete for power users: the agent does not stop at a conceptual recommendation, but produces inspectable implementation logic that connects data, controls, calculations, and visuals.

The human user can interact with Antigravity at this level by requesting changes in plain financial language. For example, the user can ask to switch from monthly to quarterly rebalancing, add a transaction-cost assumption, display the stocks that entered and exited, or show the portfolio curve as an animation. Antigravity then modifies the relevant JavaScript, HTML, and CSS so the final product reflects the desired analytical workflow.

A.3.1 Rebalancing Calendar Solver

This code block shows the operational form of a human finance instruction: rebalance monthly, quarterly, or annually. Antigravity turns that instruction into date-boundary logic that the final product can execute consistently. The calendar solver detects date boundaries (months, quarters, years) to determine when rebalancing should occur:

```
// Identify Rebalancing Dates
let rebalanceIndices = [0];
for (let d = 1; d < totalDays; d++) {
  const prevDate = dates[d - 1];
  const currDate = dates[d];

  const prevYear = prevDate.substring(0, 4);
  const currYear = currDate.substring(0, 4);
  const prevMonth = prevDate.substring(5, 7);
  const currMonth = currDate.substring(5, 7);

  let shouldRebalance = false;

  if (params.rebalanceFreq === 'monthly') {
    shouldRebalance = (prevMonth !== currMonth);
  } else if (params.rebalanceFreq === 'quarterly') {
    const prevQ = Math.floor((parseInt(prevMonth) - 1) / 3);
    const currQ = Math.floor((parseInt(currMonth) - 1) / 3);
    shouldRebalance = (prevQ !== currQ);
  } else if (params.rebalanceFreq === 'annually') {
    shouldRebalance = (prevYear !== currYear);
  }

  if (shouldRebalance) {
    rebalanceIndices.push(d);
  }
}
```

Listing A.1. Rebalancing Date Resolver in app.js

A.3.2 Dogs of the Dow Trailing Dividend Yield Solver

This implementation demonstrates how Antigravity connects raw market data to strategy selection. The human asks for dividend-yield ranking; the agent identifies the needed dividend events, price series,

lookback window, and sorting operation. Yields are computed by looking back at the preceding 365 days of dividend events for each stock relative to the current rebalance close price:

```

1 let yields = [];
2 tickers.forEach(ticker => {
3   const divEvents = dividends[ticker] || [];
4   const currentClose = closePrices[ticker][d];
5
6   // Sum dividends in last 365 days
7   const rebalanceDate = new Date(currentDateStr);
8   const oneYearAgo = new Date(rebalanceDate.getTime() - (365 * 24 * 60 * 60 *
9     1000));
10
11   let divSum = 0;
12   divEvents.forEach(e => {
13     const ed = new Date(e.date);
14     if (ed >= oneYearAgo && ed <= rebalanceDate) {
15       divSum += e.amount;
16     }
17   });
18
19   const divYield = currentClose > 0 ? divSum / currentClose : 0;
20   yields.push({ ticker, yield: divYield });
21 });
22 yields.sort((a, b) => b.yield - a.yield);
23 selectedAssets = yields.slice(0, params.stockCount).map(y => y.ticker);

```

Listing A.2. Trailing Dividend Yield Calculation in app.js

A.3.3 Transaction Cost and Order Solver

This implementation is where the final product becomes financially credible rather than merely visual. The human specifies that trading has costs and that target weights must be translated into orders; Antigravity implements the capital allocation mechanics. Calculates trade adjustments, applies costs, and solves for new share allocations:

```

1 // Compute target and current allocations
2 let targetAllocations = {};
3 tickers.forEach(ticker => {
4   targetAllocations[ticker] = weights[ticker] || 0;
5 });
6 let currentAllocations = {};
7 tickers.forEach(ticker => {
8   const stockVal = shares[ticker] * prices[ticker][d];
9   currentAllocations[ticker] = totalValue > 0 ? stockVal / totalValue : 0;
10 });
11 // Calculate trade differences
12 let tradesToExecute = [];
13 let totalTradeValue = 0;
14 tickers.forEach(ticker => {
15   const targetValue = totalValue * targetAllocations[ticker];
16   const currentValue = shares[ticker] * prices[ticker][d];
17   const tradeDiff = targetValue - currentValue;
18
19   if (Math.abs(tradeDiff) > 1.0) {
20     tradesToExecute.push({
21       ticker,

```

```

22         currentValue,
23         targetValue,
24         diff: tradeDiff,
25         price: prices[ticker][d]
26     });
27     totalTradeValue += Math.abs(tradeDiff);
28 }
29 });
30 // Deduct costs and allocate capital
31 const totalTransactionCosts = totalTradeValue * params.transactionCost;
32 const investableCapital = totalValue - totalTransactionCosts;
33 let newShares = {};
34 tickers.forEach(ticker => {
35     const tickerTargetVal = investableCapital * targetAllocations[ticker];
36     newShares[ticker] = prices[ticker][d] > 0 ? tickerTargetVal / prices[ticker][d]
37       : 0;
38 });
39 shares = newShares;
40 let finalStockValue = 0;
41 tickers.forEach(ticker => {
42     finalStockValue += shares[ticker] * prices[ticker][d];
43 });
44 cash = totalValue - finalStockValue - totalTransactionCosts;

```

Listing A.3. Order Ingestion and Execution Cost solver in app.js

A.3.4 Entrant and Exit Composition Card Appender

This code block captures the product-design refinement requested by the human user: make the portfolio changes visible at every rebalance. Antigravity responds by storing portfolio states and rendering entrants and exits as timeline cards, turning a quantitative backtest into an explanatory application. Compares target portfolios between rebalance index r and $r - 1$ to compute composition differences during playback:

```

function renderDynamicCompositionCard(rebalanceIdxIdx) {
    const rebalanceIndices = backtestResults.rebalanceIndices;
    const dailySnapshots = backtestResults.dailySnapshots;

    const currIdx = rebalanceIndices[rebalanceIdxIdx];
    const currSnapshot = dailySnapshots[currIdx];
    const currAssets = currSnapshot.assets;
    const date = currSnapshot.date;
    const portfolioVal = currSnapshot.value;

    let prevAssets = [];
    if (rebalanceIdxIdx > 0) {
        const prevIdx = rebalanceIndices[rebalanceIdxIdx - 1];
        prevAssets = dailySnapshots[prevIdx].assets;
    }

    // Set operations for composition differences
    const entrants = currAssets.filter(t => !prevAssets.includes(t));
    const exits = prevAssets.filter(t => !currAssets.includes(t));

    const card = document.createElement('div');
    card.className = 'timeline-event-card';

    // Construct HTML elements

```

```

25 const node = document.createElement('div');
26 node.className = 'timeline-node';
27 card.appendChild(node);
28
29 const header = document.createElement('div');
30 header.className = 'timeline-event-header';
31 header.innerHTML = '
32     <span class="timeline-event-date">${date}</span>
33     <span class="timeline-event-value">${Math.round(portfolioVal).
toLocaleString()}</span>
34     '
35 card.appendChild(header);
36
37 const body = document.createElement('div');
38 body.className = 'timeline-event-body';
39
40 // Entrants list rendering
41 const entrantsCol = document.createElement('div');
42 entrantsCol.className = 'timeline-column';
43 const entrantsListHTML = entrants.length === 0
44     ? '<span class="empty-column-msg">None</span>'
45     : entrants.map(t => '<span class="ticker-badge entrant">${t}</span>').join(
''');
46 entrantsCol.innerHTML = '
47     <h4>Joined</h4>
48     <div class="timeline-column-list">${entrantsListHTML}</div>
49     '
50 body.appendChild(entrantsCol);
51
52 // Exits list rendering
53 const exitsCol = document.createElement('div');
54 exitsCol.className = 'timeline-column';
55 const exitsListHTML = exits.length === 0
56     ? '<span class="empty-column-msg">None</span>'
57     : exits.map(t => '<span class="ticker-badge exit">${t}</span>').join('');
58 exitsCol.innerHTML = '
59     <h4>Left</h4>
60     <div class="timeline-column-list">${exitsListHTML}</div>
61     '
62 body.appendChild(exitsCol);
63 card.appendChild(body);
64
65 // Prepend to timeline
66 DOM.timelineFeed.insertBefore(card, DOM.timelineFeed.firstChild);
67 }

```

Listing A.4. Dynamic Entrant and Exit Badge Appender in app.js

Prompting Trail Used in this Example

This appendix shows the prompting used to create the Antigravity finance application and the accompanying report. It is included so that finance power users can see how the final product emerged through iterative human-agent interaction. The prompts are reproduced as prompts, including informal spelling, because this is part of the lesson: Antigravity can work from natural human direction and then translate that direction into structured implementation work.

B.1 Prompt 1: Establish the Financial Objective

“i want to have an algorithmic strategy to invest in the components of the dow jones”

This prompt gave Antigravity the core finance objective. The user did not specify files, functions, or libraries. Instead, the prompt identified the investable universe and the intended analytical capability. Antigravity interpreted this as a need for historical DJIA data, strategy rules, rebalancing logic, backtesting metrics, and a user-facing app.

B.2 Prompt 2: Request Dynamic Visualization

“now, i want a more interactive visualization showwing the dynamic change in composition of the portfolio...the entrants andd the exiting stocks ... show a dynamic building of the backtesiting courve”

This prompt changed the product from a static backtester into an interactive explanatory tool. Antigravity translated the human request into daily portfolio snapshots, playback controls, incremental chart updates, and entrant/exit detection. The user described the desired experience; Antigravity inferred the technical implementation.

B.3 Prompt 3: Show Every Rebalancing Event

“i want to see the changes in the porfolio composition at every rbalancing, in a dynamic display”

This prompt made the customization more precise. The user wanted every rebalancing change, not only the most recent event. Antigravity responded by creating a chronological timeline feed that records entrants and exits through the full backtest. This illustrates how review converts a working prototype into a more useful finance application.

B.4 Prompt 4: Reframe the Report

“THIS ILLUSTRATES HOW TO USE ANTIGRAVITY TO GENERATE AN APPEALING APP THAT IMPLEMENTS DIFFERENT ALGO TRADING STRATEGIES. DO NOT INCLUDE A TABLE OF CONTENTS. AUTHOR ALEJANDRO REYNOSO, JUDGE BUSINESS SCHOOL, CAMBRIDGE”

This prompt customized the documentation. Antigravity updated the title, author, affiliation, abstract framing, page headers, and removed the table of contents. This shows that Antigravity can revise not only code but also the written artifact that explains the app.

B.5 Prompt 5: Make the Audience Explicit

“EVERY SECTION MUST HAVE A DEEP EXPLANATION OF HOW ANTIGRAVITY WORKS, WHAT IS THE HUMAN INTERACTION, WHAT IS THE FINAL PRODUCT... THIS IS INTENDED TO BE AN INTRODUCTION TO ANTIGRAVITY FOR POWER USERS IN FINANCE”

This prompt shifted the report from a technical build log to a pedagogical guide. Antigravity expanded the narrative to explain how the agent works, how the human reviews and redirects it, and how the final finance app emerges from the collaboration.

B.6 Prompt 6: Require Depth, Review, and References

“EACH SECTION AT LEAST 1000 WORDS. PEDAGOGICAL, CLEAR, INTUITIVE, CAREFULLY SPEELING OUT HOW ANTIGRAVITY IS ACTUALLY WORKING. ALSO HOW ANTIGRAVITY ALLOWS THE USER TO CUSTOMIZE, IMPROVE, REVIEW. ALSO INCLUDE A BIBLIOGRAPHY WITH AT LEAST 5 REFERENCES”

This prompt added explicit acceptance criteria. Antigravity expanded the major sections, added detailed explanations of customization and review, inserted a bibliography, and compiled the LaTeX to verify that the document remained valid.

B.7 Prompt 7: Add the Prompting Appendix

“GENERATE AN APPENDIX THAT SHOWS THE PROMPTING USED IN THIS EXAMPLE”

This final prompt made the prompting process itself part of the deliverable. The appendix documents the progression from financial idea to working app to pedagogical report. It demonstrates that effective Antigravity use is iterative: the human states intent, reviews the artifact, asks for improvements, and keeps refining until the final product matches the intended purpose.

B.8 Reusable Prompting Template

Finance power users can reuse the following template:

1. State the financial objective and audience.
2. Define the desired final product: app, report, dashboard, model, or teaching case.

3. Specify data, strategies, metrics, assumptions, and constraints.
4. Describe the desired interaction: charts, playback, filters, timelines, exports, or explanatory panels.
5. Review the output and identify what is missing, unclear, or insufficiently credible.
6. Ask Antigravity to revise the code, interface, calculations, or documentation.
7. Compile, run, or inspect the final artifact before accepting it.

The main lesson is that Antigravity allows finance users to customize, improve, and review a product through natural-language prompting while preserving visible, editable artifacts.

Bibliography

- [1] Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- [2] Chan, E. P. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley.
- [3] Grinold, R. C., and Kahn, R. N. (1999). *Active Portfolio Management*. McGraw-Hill.
- [4] National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST.
- [5] Bank for International Settlements. (2024). *Artificial Intelligence in Finance*. BIS publications and working papers on AI, risk management, and financial supervision.
- [6] International Organization of Securities Commissions. (2021). *The Use of Artificial Intelligence and Machine Learning by Market Intermediaries and Asset Managers*. IOSCO.
- [7] Humble, J., and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.